PPE Compliance Detection — Hackathon Build Plan

Theme: Computer Vision • KLS VDIT Haliyal • Prepared 30 Oct 2026 • Target: working MVP in 24 hours

1. Key finding (read this first)

> **Best reference (read only, NOT cloned or copied): Purvansh09/ppe-compliance-detection (MIT).** It is the only one I found that links PPE to individual workers: it tracks each person (ByteTrack via Ultralytics) and flags them if a NO-Hardhat / NO-Safety Vest box sits mostly inside their person box, writing an annotated video plus a violations CSV. Its author lists the weaknesses, and they are exactly where your submission can win:
>
> - **Shared PPE bug:** a vest inside two overlapping people's boxes counts for both. The README proposes a fix but says it is not implemented.
> - **Containment only, no head zone:** a helmet held in hand or placed on a table, inside a person's box, can still be matched to that person.
> - Video only: no image mode, no UI, no violation summary, no hard-case handling.
> - Weak NO-* class recall (0.55 helmet, 0.61 vest) from a small, mostly augmented dataset.

Plan in one line (safe mode): write the whole project from scratch in our own structure. Use only the Ultralytics library and a public dataset (both credited). Use the repos only to learn what problems to solve: the association logic (one-to-one assignment plus head and torso zones), detecting positives (helmet, vest, person) and inferring absence, an image/video Streamlit UI with a per-worker verdict and violation summary, and low-light enhancement.

How this was researched: web search plus reading the README pages of SiteWatch, Ismailjm, hafizqaim and the Ultralytics dataset page. Other repos are listed from search snippets only and are marked **unverified**. I did not download or run any code yet.

2. Repos and what each one gives you

Repo	What exists	License	Missing / caveat
Purvansh09/ppe-compliance-detection (reference only, do not copy) github.com/Purvansh09/ppe-compliance-detection	`src/ppe_pipeline.py` (inference + ByteTrack + containment matching), `src/train.py`, `setup.ps1` for Windows, flags `--overlap`, `--require`, `--require-positive`, `--show-ppe`; outputs annotated video + violations CSV. Trained 25 epochs in about 8 min on an RTX 4060 laptop; about 24 fps at 720p. Metrics on its val set: precision 0.84, recall 0.67, mAP@0.5 0.74.	MIT. Dataset + weights: CC BY 4.0 (Roboflow "Construction Site Safety" v27).	Author-stated: shared PPE counted for overlapping workers; flicker near threshold; no re-ID; weak NO-* recall. Not stated or seen: head-zone check, image mode, UI, summary. `best.pt` not confirmed present in `models/`; you may need to train (about 8 minutes). Contents of `requirements.txt` not seen.

SiteWatch github.com/AitchEm-bot/sitewatch	Streamlit app; modules for detection, enhancement (CLAHE, denoise = helps poor lighting), fusion, drawing, CSV logging; Colab training notebook; bundled YOLOv8m weights; webcam + upload modes. **Best structural reference.**	MIT (code). YOLOv8 is AGPL-3.0; dataset has its own license.	README does not describe worker↔PPE matching. Reported recall is low (0.482), mAP@0.5 0.563 on its own model. Needs the machinery/motion parts removed.
Ultralytics Construction-PPE dataset docs.ultralytics.com/datasets/detect/construction-ppe	1,416 images (1132/143/141). Classes: helmet, gloves, vest, boots, goggles, none, Person, no_helmet, no_goggle, no_gloves, no_boots. Auto-downloads with `data="construction-ppe.yaml"`.	AGPL-3.0	No "no_vest" label (fine: we infer a missing vest from association). Small dataset. Docs page has inconsistent size figures.
Ismailjm/PPE_detection_using_YOLOV8 link	Files: `main.py`, `data_custom.yaml`, `download_img.py`, `yolov8s_custom.pt`. Helmet / vest / gloves / glasses detection on video or images.	**None stated — do not copy code**	No association logic described; dataset not named; no requirements list. Use as an idea reference only.
hafizqaim/Workspace-Safety-Detection-using-YOLOv8 link	`app.py`, `inference.py`, Kaggle training notebook, Flask-style `templates/`. Trained on Roboflow "PPE Detection v3" (>23,000 images, 17 classes) for 10 epochs.	**None stated — do not copy code**	`best.pt` must come from Releases and none is visible. No person matching. Useful as proof that the large Roboflow dataset trains in a Kaggle session.
AnshulSood11/PPE-Detection-YOLO-Deep_SORT link	Per-person IDs across frames with Deep SORT, alert if a person lacks PPE for a set time. (snippet only, unverified)	Unverified	Marked work-in-progress, uses YOLOv3. Reference for the optional video-tracking feature; we would use Ultralytics' built-in tracker instead.
VoxDroid/Construction-Site-Safety-PPE-Detection link	YOLOv8, 10 classes (Hardhat, Safety Vest, NO- variants), Flask + SocketIO. (snippet only, unverified)	Unverified	Check README for license and any association code before relying on it.
Ansarimajid/Construction-PPE-Detection link	Helmet / vest / mask, FastAPI, Docker, email alerts. (snippet only, unverified)	Unverified	Reference for alerting / API packaging only; not needed for MVP.
ZijianWang-ZW/PPE_detection link	Research repo, 8 YOLO models, open dataset with persons, vests, helmet colors; best reported mAP 86.55% (YOLOv5x). (snippet only, unverified)	Unverified	Possible extra dataset source. Check license before use.

darshan1654/ai-ppe-detection and **arshvir HF Space**	Streamlit dashboard with compliance logs/reports; HF Space with helmet/vest/gloves/boots. (snippet only, unverified)	Unverified	Dashboard layout ideas.

Datasets on Roboflow Universe (check each page's license and image count)

Dataset	Labels (per search snippet)
PPE - detection3	helmet, vest, person, no-helmet, no-vest. Page lists CC BY 4.0. **Best match for our 3 classes.**
ppe_master	helmet, vest, person
PPECustomV3	Helmet, Vest, Person, Chin Strap, Ear Muff
ppe-not-human	boots, helmets, people with/without PPE, hi-vis vests
Roboflow PPE blog tutorial	Walkthrough using a person/helmet/vest/no-helmet/no-vest dataset

> **Gap in the data:** I did not find a dataset that includes the problem statement's hard cases — a helmet in hand or on a table, and hi-vis clothing that is not a vest. You will need to collect 30–50 such images yourselves (phone photos or web images) and use them as a **test set**, plus label a handful as hard negatives if time allows.

3. Feature checklist: what exists vs what you build

Requirement from the problem statement	Status	Where it comes from
Detect people, helmets, safety vests	Exists	Pretrained YOLO + Roboflow / Ultralytics dataset; fine-tune on your GPU
Short video clips (optional)	Partly	We write the frame loop ourselves with Ultralytics' built-in `model.track()` (a documented library feature), plus our own annotated video and CSV output
Associate each item with the person wearing it	Build	Our own code: one-to-one assignment with head/torso zones (section 5). Purvansh09's containment idea is only a reference for the problem.
Predefined compliance rules per worker	Build	Our code: small config (e.g. `required = [helmet, vest]`)
Flag each non-compliant worker + missing item	Build	Our code, output of the rules engine
Annotated images with per-worker verdict	Partly	SiteWatch / Ultralytics drawing for boxes; our own per-worker labels and colors
Violation summary	Partly	SiteWatch's CSV logging as a pattern; our own summary table and JSON/CSV export
Multiple workers, occlusion, distance, poor lighting	Partly	CLAHE (SiteWatch idea), larger `imgsz`, augmentation, tiled inference for small far workers, tuned thresholds
Helmet in hand / on table must not count	Build	Head-zone constraint in association + your hard-case test set
Hi-vis clothing that is not a vest	Build	Hard-negative images + vest confidence threshold + torso-coverage check

4. Setup (do this in the first hour)

> Your machine has Python 3.14 and an RTX 5060 (8 GB). PyTorch and Ultralytics may not support 3.14, and the 5060 needs a CUDA 12.8 or newer PyTorch build. Use a Python 3.12 virtual environment. `pip` is not on your PATH, so use `python -m pip`.

```
py -3.12 -m venv .venv          # install Python 3.12 first if this fails
.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128
python -m pip install ultralytics opencv-python supervision streamlit pandas pillow
python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
```

If the last line prints `False`, stop and fix CUDA before anything else; training on CPU will not finish in time.

5. The part we build: association and rules

Purvansh09's repo flags a person when a NO-* box sits “mostly inside” their box. Our own algorithm below solves the two weaknesses its author describes. It is written from scratch in our `ppe/` modules; nothing is copied from that repo.

Algorithm

```
for each person box P:
    head_zone  = top 35% of P
    torso_zone = P between 20% and 65% of height

candidate pairs (person, helmet): helmet center inside head_zone AND
        overlap(helmet, head_zone) / area(helmet) >= 0.5
candidate pairs (person, vest):   vest center inside torso_zone AND
        overlap(vest, P) / area(vest) >= 0.6

score each pair (overlap + distance to zone center), then assign
greedily best-first so one helmet / vest goes to ONE person only
(if two people overlap, the closer head wins)

verdict per person:
    missing = [item for item in REQUIRED if item not assigned]
    compliant = (missing == [])
```

Why this satisfies the constraint

- A helmet on a table or in a hand is outside every head zone, so it is never assigned, so the worker is flagged **NO HELMET**.
- One helmet cannot validate two workers because each item is assigned to only one person.
- A person whose head is cropped out of the frame gets “helmet unverifiable” rather than a false pass (optional refinement: add an `uncertain` status).

Suggested project layout

```
ppe-detection/
  app.py                 # Streamlit UI
  ppe/detector.py        # YOLO wrapper (person, helmet, vest)
  ppe/associate.py       # OUR matching logic (section 5)
  ppe/rules.py           # required items config + verdicts
  ppe/annotate.py        # green / red boxes, per-worker labels
  ppe/summary.py         # counts, per-violation table, CSV / JSON export
  ppe/enhance.py         # CLAHE for low light (optional)
  tests/test_associate.py# unit tests with hand-made boxes
  data/                  # dataset yaml, hard-case test images
  runs/                  # training output
  README.md              # includes CREDITS section
```

6. Step-by-step from scratch (24 hours)

Hours	Task	Details / done-when
0–1	Environment	Section 4. Done when CUDA prints True.
1–2	Dataset	Download Roboflow "PPE - detection3" in YOLOv8 format (or use Ultralytics Construction-PPE). Keep only person, helmet, vest; drop the "no-" classes so the model learns positives and we infer absence. Check licenses.
2–3	Baseline training	`yolo detect train model=yolo11s.pt data=data.yaml imgsz=960 epochs=60 batch=16` (lower the batch if memory runs out). Starts in the background.
2–8	Write while training runs	`associate.py`, `rules.py` and unit tests using made-up boxes (helmet on table, two workers one helmet, etc.). This is the core logic and can be developed without the model.
3–4	Hard-case test set	Collect 30–50 images: compliant, missing helmet, missing vest, helmet in hand, helmet on table, orange t-shirt, crowded, far away, dark. Keep them **out of training**.
6–9	Plug model into pipeline	Detector → associate → rules → annotate. Run on the hard-case set and note failures.
9–13	Streamlit UI + summary	Upload image or clip; show annotated output, per-worker table, violation summary, and CSV / JSON download.
13–17	Robustness	Larger `imgsz` or tiled inference for distant workers, CLAHE for dark images, confidence and zone thresholds tuned on the hard-case set, augmentation retrain if needed.
17–20	Video (optional)	Frame loop with `model.track()` so each worker keeps an ID; report a violation only if seen in N frames to avoid flicker.
20–22	Evaluation numbers	Report detection mAP plus per-worker verdict accuracy on your hard-case set. Judges care about the verdict accuracy more than raw mAP.
22–24	Docs and demo	README with CREDITS, architecture diagram, 5 demo images (including a failure case you explain honestly), 2–3 minute pitch.

7. Licensing and "not cheating" checklist

> **Safe mode rules for this project**
>
> 1. No `git clone`, fork, or copy-paste of any repo. Every source file is written by us.
> 2. The only outside code is the **Ultralytics** library and **OpenCV / Streamlit / pandas**, installed with pip, which is normal.
> 3. Training uses one **public dataset** whose license we have checked, credited in the README.
> 4. The README lists the **repos we studied for ideas** under "References", with what we learned (for example "containment-based matching has a shared-PPE flaw"). Being open about this is what keeps it safe.
> 5. Commit as we go, so the git history shows the project being built step by step.
> 6. If a judge asks, the honest answer is: "we used YOLO and a public dataset for detection, and wrote the association, rules, summary and UI ourselves."

- **Read the hackathon rules** for any "from scratch" or "no pre-existing code" clause. If it exists, use the repos for ideas only.
- **Do not copy code from repos with no license** (Ismailjm, hafizqaim). Read their approach, write your own.
- Purvansh09's repo and SiteWatch are both MIT, but we are **not reusing their code**. Their datasets and weights are CC BY 4.0 / mixed; if we train on the same Roboflow dataset, credit its author.
- Ultralytics YOLO and the Construction-PPE dataset are **AGPL-3.0**. Fine for a demo and open-source submission; mention it in the README. It matters only if you later distribute a closed product.
- Roboflow datasets have per-dataset licenses (detection3 shows CC BY 4.0). Cite author and dataset.
- README **CREDITS** section: every repo, dataset and model used, with link, license, and exactly what you took from it.
- Be honest in the pitch: "detection from a pretrained YOLO and public dataset; association, rules, summary and robustness handling are our work."

8. Risks and fallbacks

Risk	Fallback
CUDA / PyTorch won't install on the 5060	Train on free Kaggle or Colab GPU (SiteWatch and hafizqaim both did this), run inference locally on CPU.
Weak vest or helmet recall	Merge a second Roboflow dataset; lower confidence threshold; use `imgsz=1280`; tile small images.
Hi-vis non-vest triggers false "vest"	Add hard negatives and raise the vest confidence threshold; require vest box to cover a minimum share of the torso zone.
Person box misses occluded workers	Lower the person threshold; mark the worker "uncertain" instead of passing them.
Running out of time	Ship the image-only MVP (sections 5–6 up to hour 13). Video is optional in the problem statement.

9. Sources

- Purvansh09/ppe-compliance-detection • AitchEm-bot/sitewatch • Ismailjm/PPE_detection_using_YOLOV8 • hafizqaim/Workspace-Safety-Detection-using-YOLOv8 • AnshulSood11/PPE-Detection-YOLO-DeepSORT

- VoxDroid/Construction-Site-Safety-PPE-Detection • Ansarimajid/Construction-PPE-Detection • ZijianWang-ZW/PPE_detection • darshan1654/ai-ppe-detection
- Ultralytics Construction-PPE dataset docs • Roboflow PPE detection blog